

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO1: *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks: 25

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

*I SAHASRA V(192424234) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: SAHASRA V

SET 2

1. A machine learning model shows the following loss values during training: (5 Marks) (a) Epoch 1: Training Loss = 3.0, Validation Loss = 2.7 (b) Epoch 2: Training Loss = 2.6, Validation Loss = 2.3 (c) Epoch 3: Training Loss = 2.3, Validation Loss = 2.1 (d) Epoch 4: Training Loss = 2.1, Validation Loss = 2.0 (e) Epoch 5: Training Loss = 1.9, Validation Loss = 2.2 (f) Epoch 6: Training Loss = 1.7, Validation Loss = 2.5 (g) Epoch 7: Training Loss = 1.6, Validation Loss = 2.8 (h) Epoch 8: Training Loss = 1.4, Validation Loss = 3.1

i) At which epoch does the model achieve optimal generalization?

ii) Identify signs of overfitting from the data.

iii) Recommend two regularization techniques and explain their impact

CODE:

```
loss.py - C:/Users/hp/Downloads/loss.py (3.13.9)
File Edit Format Run Options Window Help
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Epoch": [1, 2, 3, 4, 5, 6, 7, 8],
    "Training_Loss": [3.0, 2.6, 2.3, 2.1, 1.9, 1.7, 1.6, 1.4],
    "Validation_Loss": [2.7, 2.3, 2.1, 2.0, 2.2, 2.5, 2.8, 3.1]
}

df = pd.DataFrame(data)

print("Dataset:")
print(df)

best_epoch = df.loc[df["Validation_Loss"].idxmin(), "Epoch"]

print("\n(i) Optimal Generalization Epoch:", best_epoch)

print("\n(ii) Signs of Overfitting:")
print("After Epoch 4, the training loss continues to decrease while the validation loss increases, indicating overfitting.")

print("\n(iii) Regularization Techniques:")
print("1. Dropout - Randomly disables neurons during training to reduce overfitting.")
print("2. L2 Regularization - Adds a penalty to large weights to improve generalization.")

plt.figure(figsize=(8,5))
plt.plot(df["Epoch"], df["Training_Loss"], marker='o', label="Training Loss")
plt.plot(df["Epoch"], df["Validation_Loss"], marker='s', label="Validation Loss")

plt.title("Training Loss vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.xticks(df["Epoch"])
plt.grid(True)
plt.legend()

plt.show()
```

OUTPUT:

```
*IDLE Shell 3.13.9*
File Edit Shell Debug Options Window Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/hp/Downloads/loss.py =====
Dataset:
Epoch Training_Loss Validation_Loss
0 1 3.0 2.7
1 2 2.6 2.3
2 3 2.3 2.1
3 4 2.1 2.0
4 5 1.9 2.2
5 6 1.7 2.5
6 7 1.6 2.8
7 8 1.4 3.1

(i) Optimal Generalization Epoch: 4

(ii) Signs of Overfitting:
After Epoch 4, the training loss continues to decrease while the validation loss starts increasing.

(iii) Regularization Techniques:
1. Dropout - Randomly disables neurons during training to reduce overfitting.
2. L2 Regularization - Adds a penalty to large weights to improve generalization
.
```



2. In an industrial process, an AI robot is assigned to segregate the front and back tires automatically. Let the number of tires are 250 (125 front and 125 back tires). (5 Marks) a. Front identified as Front: 119 a. Front identifies as Back: 06 a. Back identified as Back: 120 a. Back identified as Front: 05 Compute: Accuracy, Precision, Sensitivity, Specificity and F1-Score.

CODE:

```
TP.py - C:/Users/hp/Downloads/TP.py (3.13.9)
File Edit Format Run Options Window Help
TP = 119
TN = 120
FP = 5
FN = 6

accuracy = (TP + TN) / (TP + TN + FP + FN)

precision = TP / (TP + FP)

sensitivity = TP / (TP + FN)

specificity = TN / (TN + FP)

f1_score = 2 * (precision * sensitivity) / (precision + sensitivity)

print("Accuracy      :", round(accuracy, 4))
print("Precision     :", round(precision, 4))
print("Sensitivity    :", round(sensitivity, 4))
print("Specificity    :", round(specificity, 4))
print("F1-Score       :", round(f1_score, 4))

print("\nAccuracy (%) :", round(accuracy * 100, 2), "%")
print("Precision (%) :", round(precision * 100, 2), "%")
print("Sensitivity (%) :", round(sensitivity * 100, 2), "%")
print("Specificity (%) :", round(specificity * 100, 2), "%")
print("F1-Score (%) :", round(f1_score * 100, 2), "%")
```

OUTPUT:

```
IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/hp/Downloads/TP.py =====
Accuracy      : 0.956
Precision     : 0.9597
Sensitivity    : 0.952
Specificity    : 0.96
F1-Score       : 0.9558

Accuracy (%) : 95.6 %
Precision (%) : 95.97 %
Sensitivity (%) : 95.2 %
Specificity (%) : 96.0 %
F1-Score (%) : 95.58 %
...
```

3. In a medical diagnosis system, an AI model detects whether a patient has a disease or not. Out of 300 cases (150 diseased and 150 healthy): a. Diseased identified as Diseased: 138 a. Diseased identified as Healthy: 12 a. Healthy identified as Healthy: 135 a. Healthy identified as Diseased: 15 Compute: Accuracy, Precision, Sensitivity (Recall), Specificity, and F1-Score

CODE:

```
MEDICAL.py - C:/Users/hp/Downloads/MEDICAL.py (3.13.9)
File Edit Format Run Options Window Help
TP = 138
TN = 135
FP = 15
FN = 12

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)
f1_score = 2 * (precision * sensitivity) / (precision + sensitivity)

print("Accuracy      :", round(accuracy, 4))
print("Precision     :", round(precision, 4))
print("Sensitivity    :", round(sensitivity, 4))
print("Specificity    :", round(specificity, 4))
print("F1-Score       :", round(f1_score, 4))

print("\nAccuracy (%) :", round(accuracy * 100, 2), "%")
print("Precision (%) :", round(precision * 100, 2), "%")
print("Sensitivity (%) :", round(sensitivity * 100, 2), "%")
print("Specificity (%) :", round(specificity * 100, 2), "%")
print("F1-Score (%) :", round(f1_score * 100, 2), "%")
```

OUTPUT:

```
File Edit Shell Debug Options Window Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/hp/Downloads/MEDICAL.py =====
Accuracy      : 0.91
Precision     : 0.902
Sensitivity    : 0.92
Specificity    : 0.9
F1-Score       : 0.9109

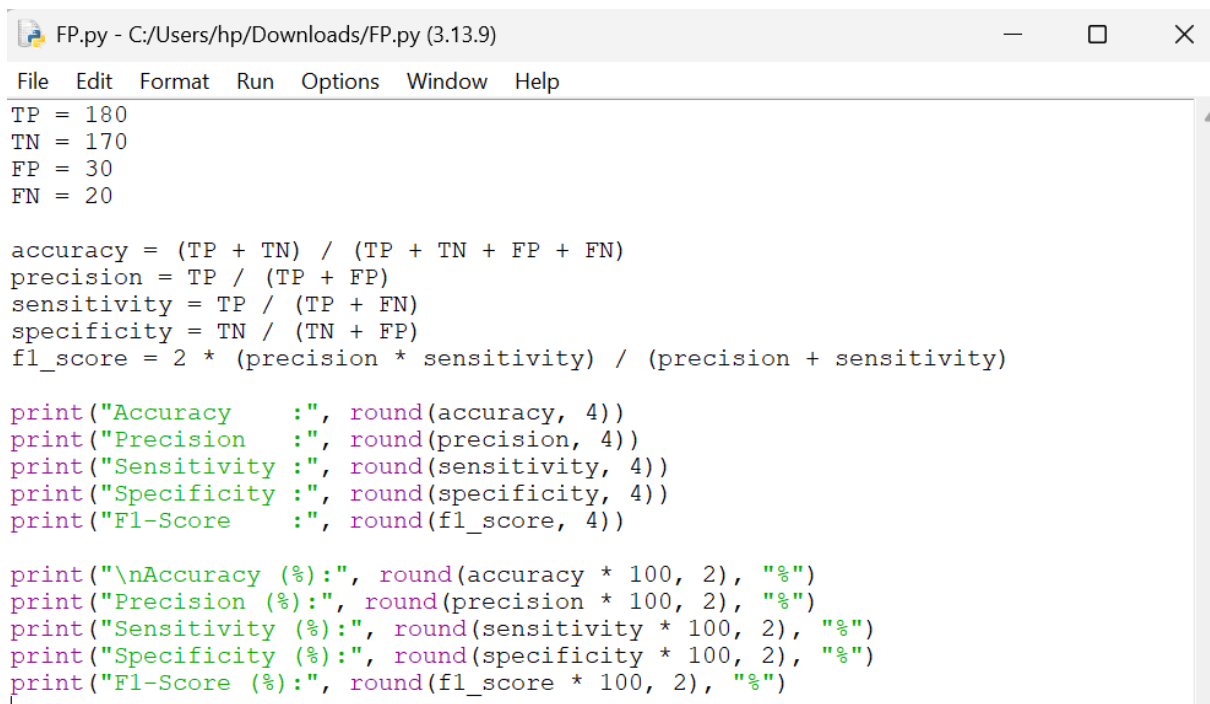
Accuracy (%): 91.0 %
Precision (%): 90.2 %
Sensitivity (%): 92.0 %
Specificity (%): 90.0 %
F1-Score (%): 91.09 %
>>>
```

4. In a spam email filtering system, an AI model classifies emails as Spam or Not Spam. Out of 400 emails (200 spam and 200 non-spam):

- ☐ Spam identified as Spam: 180
- ☐ Spam identified as Not Spam: 20
- ☐ Not Spam identified as Not Spam: 170
- ☐ Not Spam identified as Spam: 30

Compute: Accuracy, Precision, Sensitivity, Specificity, and F1-Score.

CODE:



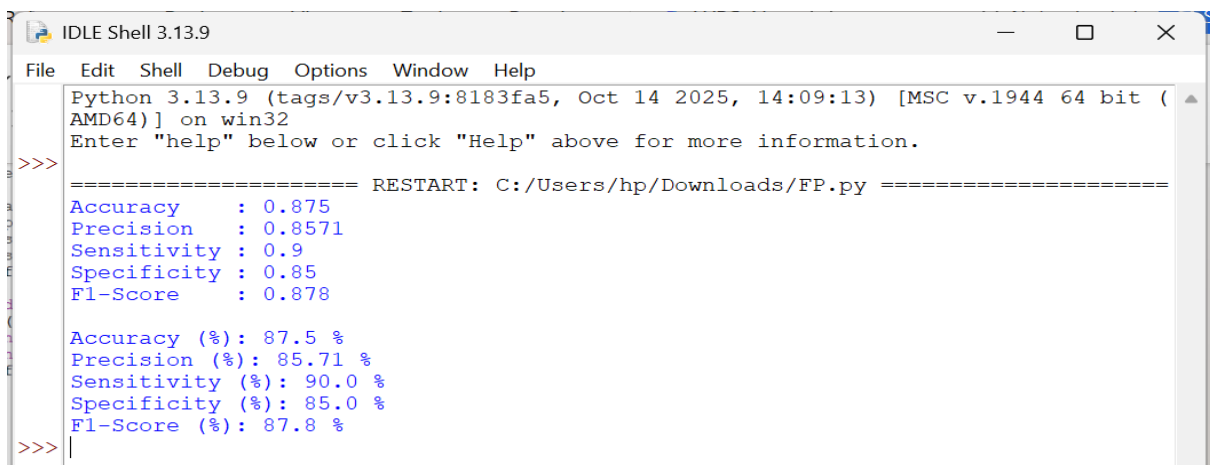
```
FP.py - C:/Users/hp/Downloads/FP.py (3.13.9)
File Edit Format Run Options Window Help
TP = 180
TN = 170
FP = 30
FN = 20

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)
f1_score = 2 * (precision * sensitivity) / (precision + sensitivity)

print("Accuracy      :", round(accuracy, 4))
print("Precision     :", round(precision, 4))
print("Sensitivity    :", round(sensitivity, 4))
print("Specificity    :", round(specificity, 4))
print("F1-Score      :", round(f1_score, 4))

print("\nAccuracy (%):", round(accuracy * 100, 2), "%")
print("Precision (%):", round(precision * 100, 2), "%")
print("Sensitivity (%):", round(sensitivity * 100, 2), "%")
print("Specificity (%):", round(specificity * 100, 2), "%")
print("F1-Score (%):", round(f1_score * 100, 2), "%")
```

OUTPUT:



```
IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/hp/Downloads/FP.py =====
Accuracy      : 0.875
Precision     : 0.8571
Sensitivity    : 0.9
Specificity    : 0.85
F1-Score      : 0.878

Accuracy (%): 87.5 %
Precision (%): 85.71 %
Sensitivity (%): 90.0 %
Specificity (%): 85.0 %
F1-Score (%): 87.8 %

>>>
```

5. In a quality inspection system in manufacturing, an AI model identifies defective

and non-defective products. Out of 500 products (250 defective and 250 non-defective): (5

Marks) Defective identified as Defective: 230

Defective identified as Non-Defective: 20

Non-Defective identified as Non-Defective: 225

Non-Defective identified as Defective: 25

Compute: Accuracy, Precision, Sensitivity, Specificity, and F1-Score.

CODE:

```
ROUND.py - C:/Users/hp/Downloads/ROUND.py (3.13.9)
File Edit Format Run Options Window Help
TP = 230
TN = 225
FP = 25
FN = 20

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)
f1_score = 2 * (precision * sensitivity) / (precision + sensitivity)

print("Accuracy      :", round(accuracy, 4))
print("Precision     :", round(precision, 4))
print("Sensitivity    :", round(sensitivity, 4))
print("Specificity    :", round(specificity, 4))
print("F1-Score      :", round(f1_score, 4))

print("\nAccuracy (%) :", round(accuracy * 100, 2), "%")
print("Precision (%) :", round(precision * 100, 2), "%")
print("Sensitivity (%) :", round(sensitivity * 100, 2), "%")
print("Specificity (%) :", round(specificity * 100, 2), "%")
print("F1-Score (%) :", round(f1_score * 100, 2), "%")
```

OUTPUT:

```
IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/hp/Downloads/ROUND.py =====
Accuracy      : 0.91
Precision     : 0.902
Sensitivity    : 0.92
Specificity    : 0.9
F1-Score      : 0.9109

Accuracy (%) : 91.0 %
Precision (%) : 90.2 %
Sensitivity (%) : 92.0 %
Specificity (%) : 90.0 %
F1-Score (%) : 91.09 %
>>>
```